[Khanh Pham] - Toolchain & Cross-Compilation

Toolchain and cross-compilation

Overview

As we know, we must convert from programming file (.c) to execute file to run. A toolchain is the complete set of tools required to transform source code into executables for a target architecture.

There are 2 types of toolchain:

- Native toolchain: The executing file directly run on the main device
- Cross toolchain: The executing file can run on the other architecture device

If we run by the executing file which build by not true toolchain, the result can be like this: (x86-64 to aarch64)

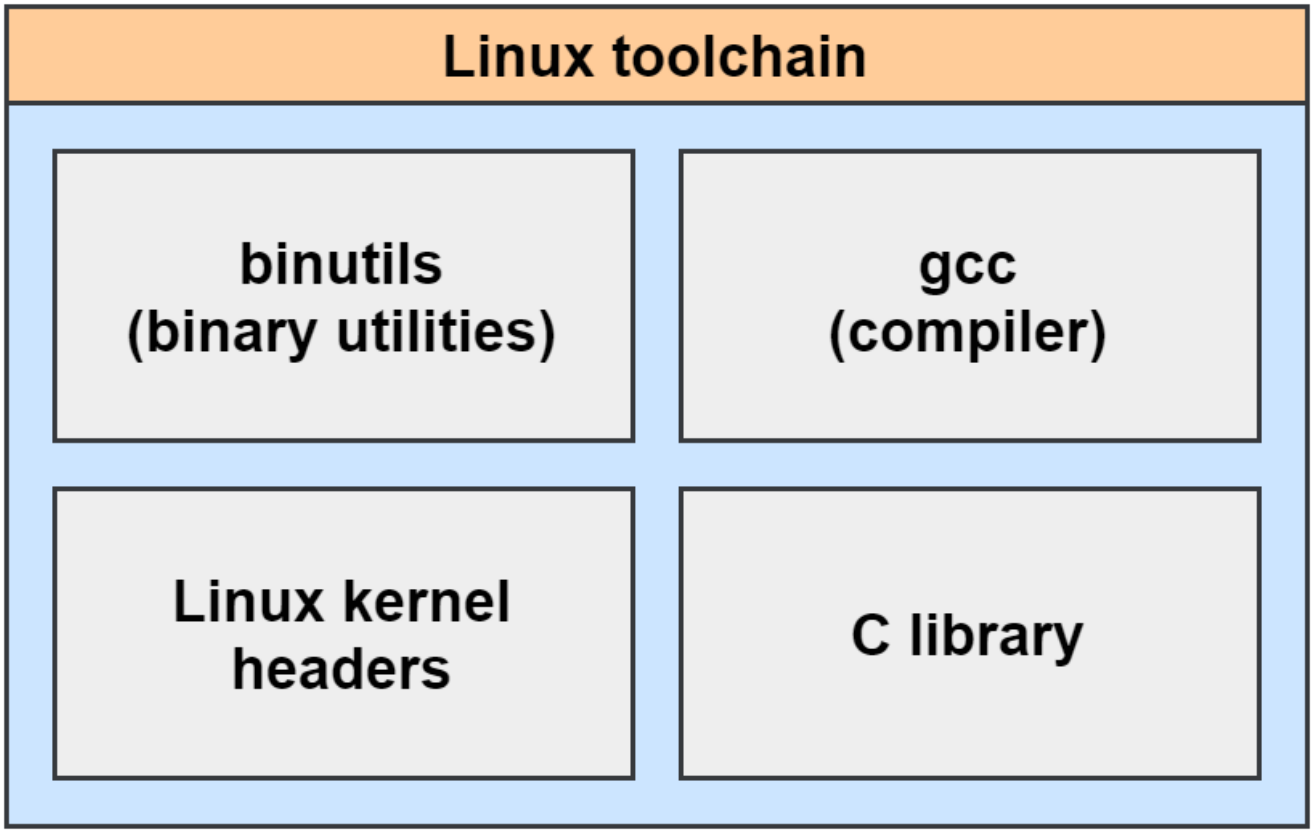
```
-bash: ./test: cannot execute binary file: Exec format error
```

> So, to make this file can run on other architecture device, we need build custom toolchain for the main device.

Before going to how to create custom toolchain, we must undertand inside the toolchain:

Toolchain

There are 4 main parts of a toolchain:



- `Binutils`: This component consist the step when comparing from asembly code to object code and linking
- `GCC compiler`: This component consist the step when comparing from C language code to assembly code
- `Kernel headers`: This component consist set of systemcall that userspace can use to communicate with kernel. When we choose the version of kernel header, it must older than the version of kernel on the device.
- `C lib`: This is very important component, it defines and provides higher function api for application.

In C lib, there are some types:

	glibC	**musl**	**uClibC**
Description	Standard lib C, provide rich and robust set of C language functions	Containers, Lightweight Linux, Modern Embedded	A small standard lib C, Custom for each system
Size	Large	Small	Very small

> Now we know the executing file can only run on the suitable device which build by toolchain. That mean if we want to create a exe file for ARM chip

> architecture, we must use toolchain of ARM

Let go to the previous example: If we want to the executing file that build by x86_64 can run on ARM, we must use the ARM toolchain.

I will use aarch64-linux-gnu toolchain to build, after that, when we move the exe file to ARM device, the application successful run.

The rule of toolchain name:

```
<target>-<vendor>-<os>-<abi>
```

- `Target`: The target architecture
- `Vendor`: The vendor of toolchain
- `OS`: The suitable OS of toolchain
- `abi`: Rule of argument, lib C of this toolchain

There are 2 ways of toolchain that we can use: Ready-to-use cross-compiling toolchains and building a custom toolchain

With Ready-to-use toolchain, we just need to download and use, it's really easy

With custom toolchain, we must use the Crosstool-NG to build.

> When we set the kernel header of toolchain, it have not to newer than kernel version of target board because if board have older kernel version, there are some new system call definitions in kernel header of toolchain. Therefore, if we use this systemcalls, the toolchain will not know about them.

What happen if we use wrong toolchain?

When we build the exe file which build by wrong toolchain, the error code message will end with:

```
No such file or directory
```

The file here is not exe file, it is the shared lib file(.so) for each architecture. For example:

-x86_64: /lib/ld-linux.so.2

- aarch64: /lib/ld-linux-aarch64.so.1

When we build the exe file with a specific architecture, in ELF header section will contain suitable of shared lib dir:

For example, Reading the ELF section of aarch64 exe file by command: `readelf -l'

```
INTERP         0x0000000000000238 0x0000000000400238 0x0000000000400238
                 0x000000000000001b 0x000000000000001b  R      0x1
      [Requesting program interpreter: /lib/ld-linux-aarch64.so.1]
  LOAD           0x0000000000000000 0x0000000000400000 0x0000000000400000
                 0x00000000000007cc 0x00000000000007cc  R E    0x10000
  LOAD           0x000000000000fdd8 0x000000000041fdd8 0x000000000041fdd8
                 0x0000000000000258 0x0000000000000260  RW     0x10000
```

As we can see, this exe file request the shared lib file of aarch64 architecture.

If we want to build and run the exe file of another kernel library like from gnu to uClibC, we must use the flag `-static` when build exe file. After that, the exe file will contain all necessary function definition inside without using it in shared lib.

Revision #3
Created 2026-10-20 04:05:05 UTC by Khanh Pham
Updated 2026-10-21 10:26:18 UTC by Khanh Pham